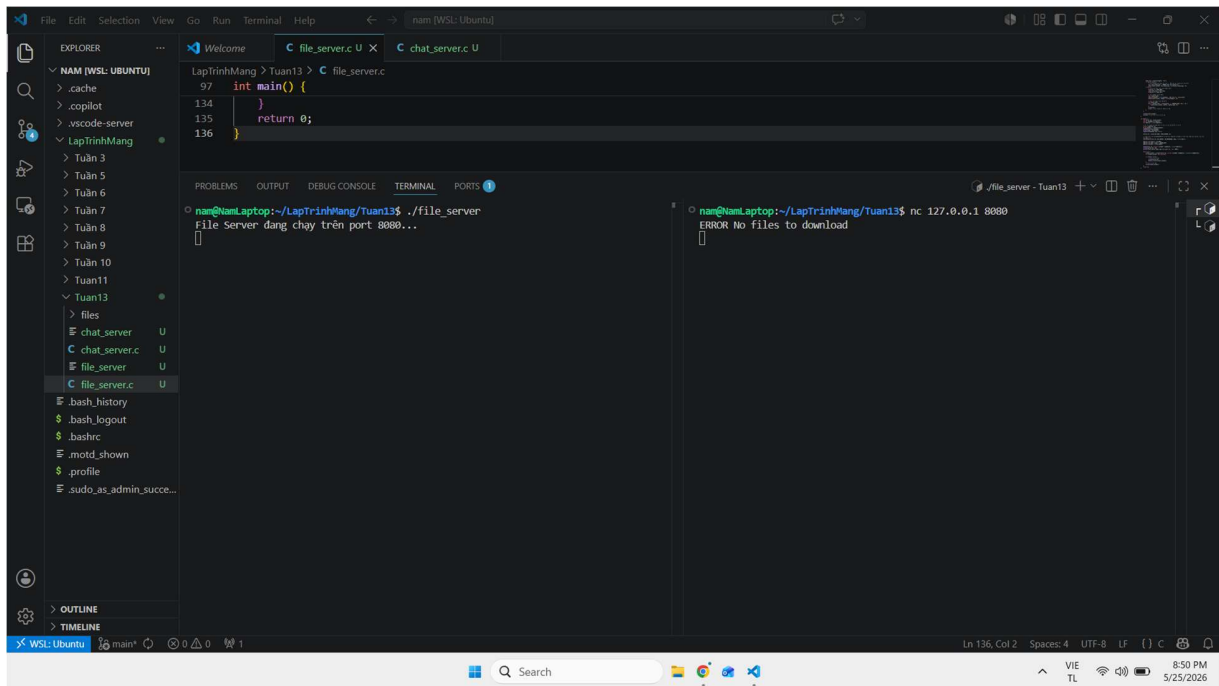


Báo cáo bài tập về nhà tuần 13

Họ và tên: Nguyễn Thành Nam

MSSV: 20225368

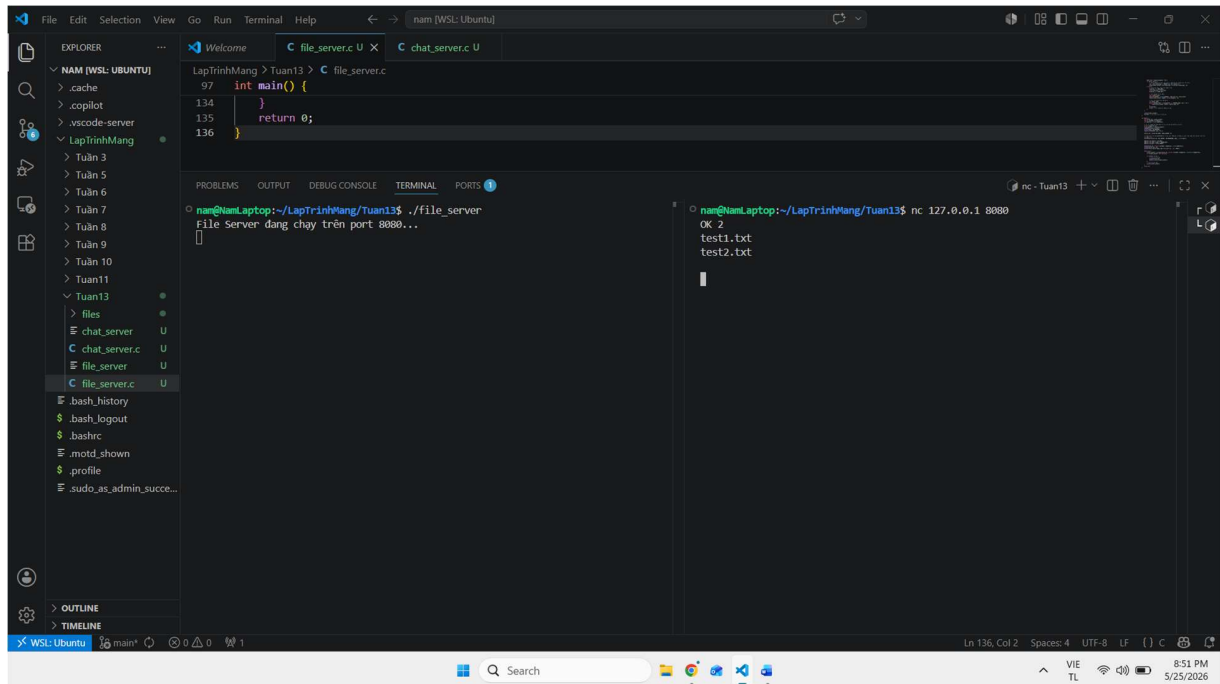
Bài 3.1:pp



Hình 1. Xử lý khi thư mục Server rỗng

Giải thích kết quả:

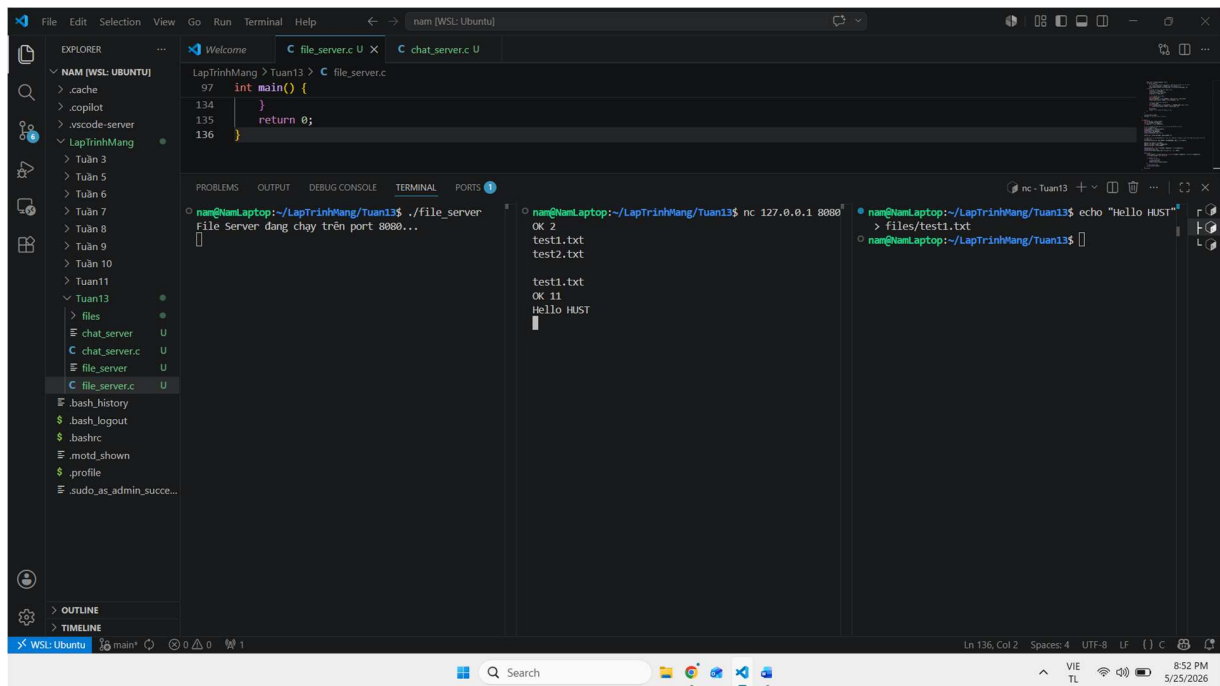
- **Phía Server (Terminal trái):** Khởi tạo thành công Socket TCP, bind vào cổng 8080 và lắng nghe kết nối từ các Client.
- **Phía Client (Terminal phải):** Sử dụng công cụ nc (Netcat) để kết nối đến Server. Do thư mục ./files trên hệ thống lúc này chưa có tệp tin nào (\$N=0\$), tiến trình con của Server ngay lập tức phản hồi chuỗi kiểm tra đúng định dạng giao thức: ERROR No files to download\r\n.
- **Kết quả:** Ngay sau khi gửi thông báo lỗi, Server chủ động gọi hàm close() để giải phóng Socket, Client lập tức bị ngắt kết nối an toàn (quay về dấu nhắc dòng lệnh).



Hình 2. Server phản hồi danh sách tệp tin cho Client khi kết nối thành công

Giải thích kết quả:

- Sau khi quản trị viên tạo thêm 2 tệp tin cấu hình nhị phân thử nghiệm là test1.txt và test2.txt trong thư mục hệ thống.
- Khi Client thực hiện kết nối lại, tiến trình con (Child Process) được tách ra từ tiến trình cha thông qua cơ chế fork() đã quét thư mục và đếm được $N=2$.
- Kết quả:** Server phản hồi đúng chuẩn dữ liệu: Dòng đầu tiên gửi mã trạng thái OK 2\r\n, theo sau là danh sách tên các file phân tách bằng ký tự xuống dòng \r\n. Khác với kịch bản 1, lúc này Server giữ trạng thái kết nối ổn định và chuyển sang chế độ block để đợi nhận lệnh yêu cầu tải tệp tin từ phía Client.

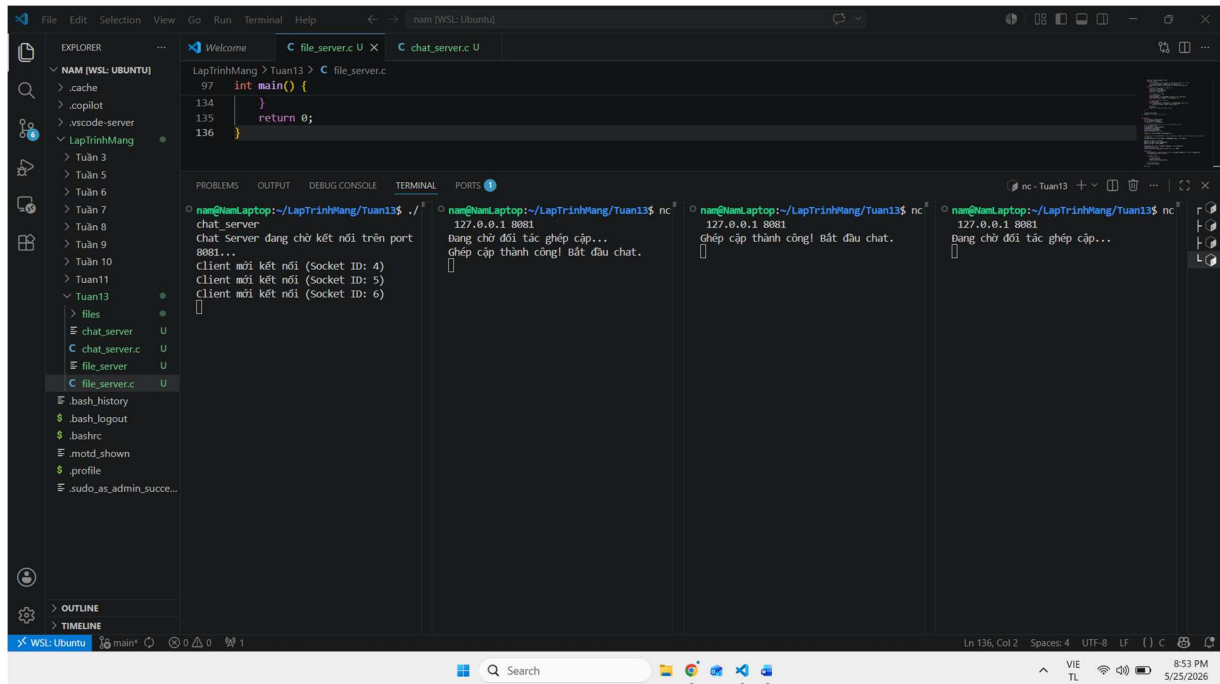


Hình 3. Luồng gửi nhận dữ liệu và truyền tải nội dung tệp tin nhị phân

Giải thích kết quả:

- **Terminal phải (Quản trị):** Khởi tạo nội dung cho file bằng câu lệnh echo "Hello HUST" > files/test1.txt. Kích thước tệp tin bao gồm cả ký tự xuống dòng được hệ thống xác định là 11 bytes.
- **Terminal giữa (Client):** Client gửi chuỗi request chứa chính xác tên file cần lấy là test1.txt.
- **Kết quả:** Server kiểm tra thấy file hợp lệ, đọc metadata để lấy kích thước dữ liệu và phản hồi lại header kích thước theo cấu trúc OK 11\r\n. Tiếp theo đó, luồng dữ liệu nhị phân chứa chuỗi văn bản "Hello HUST" được đẩy liên tục qua luồng Stream sang buffer của Client. Truyền tải hoàn tất, tiến trình con đóng Socket kết nối đồng bộ ở cả hai đầu.

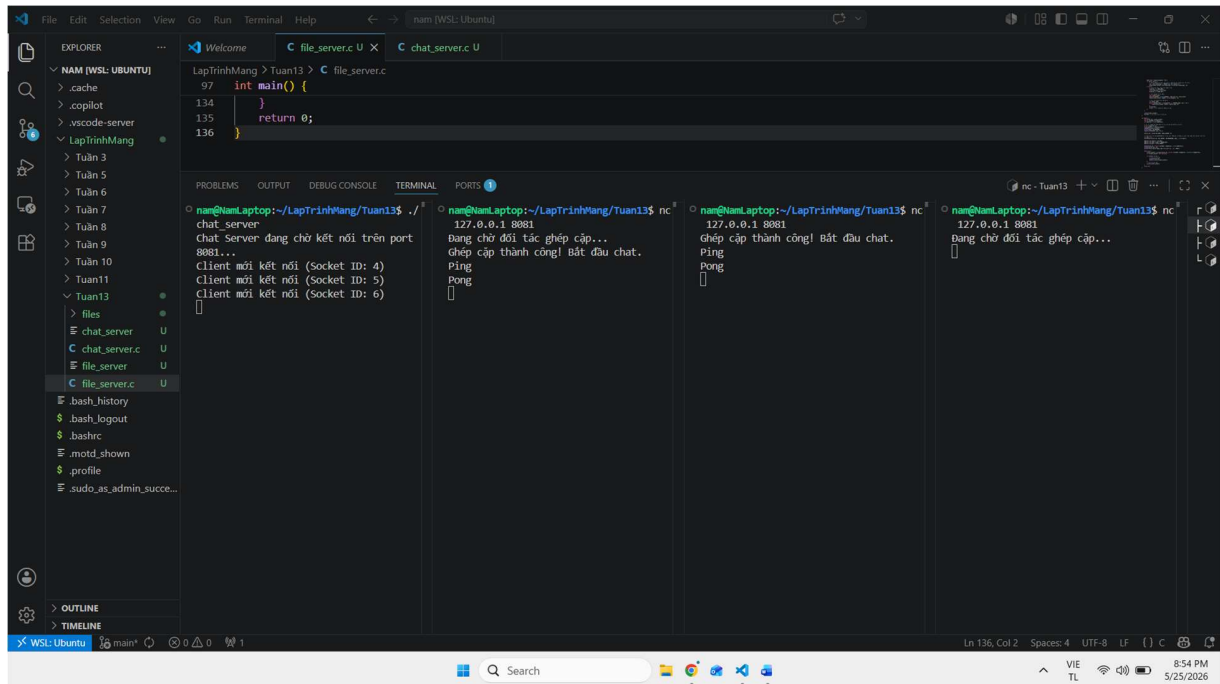
Bài 3.2:



Hình 4. Cơ chế Multi-threading kiểm soát hàng đợi (Queue) và thiết lập phiên (Session).

Giải thích kết quả:

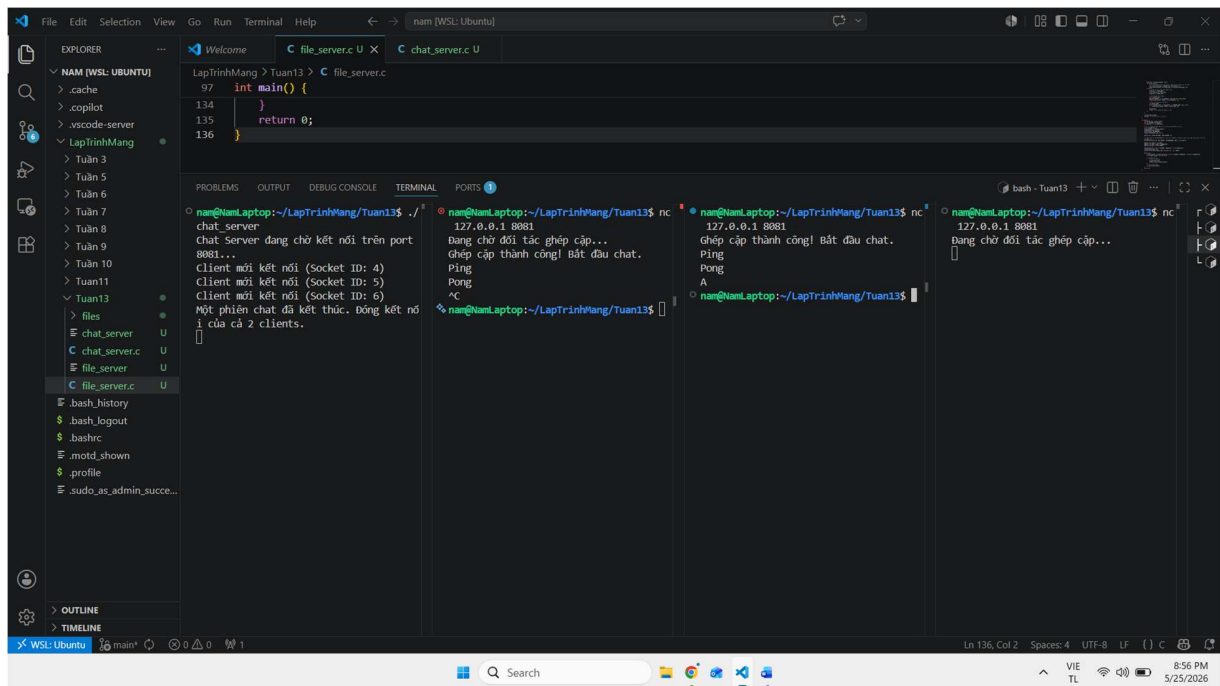
- **Terminal 1 (Server):** Khởi chạy Chat Server đa luồng tại cổng 8081. Hệ thống bắt đầu log lại Socket ID của từng kết nối để quản lý tranh chấp tài nguyên bằng cơ chế Mutex Lock.
- **Terminal 2 (Client 1 - ID: 4):** Kết nối đầu tiên, do biến hàng đợi `waiting_client == -1`, Client 1 nhận thông báo trạng thái Đang chờ đối tác ghép cặp... và được đưa vào trạng thái treo (sleep).
- **Terminal 3 (Client 2 - ID: 5):** Kết nối thứ hai, Server nhận diện hàng đợi đã đủ điều kiện ghép đôi (\$\ge 2\$). Tiến trình gọi hàm giải phóng biến đợi, khởi tạo cấu trúc struct chứa cặp Socket và sinh ra một Thread độc lập (`pthread_create`) để quản lý phiên. Cả 2 Client đồng thời nhận thông báo: Ghép cặp thành công! Bắt đầu chat..
- **Terminal 4 (Client 3 - ID: 6):** Khi Client 3 kết nối vào hệ thống, do cặp trước đã được xử lý tách luồng riêng biệt, Client 3 tiếp tục được đẩy vào một hàng đợi mới một cách an toàn và đứng chờ Client tiếp theo.



Hình 5. Quá trình truyền thông điệp tương tác thời gian thực giữa hai luồng Socket.

Giải thích kết quả:

- Trong hàm xử lý phiên chat (session_handler), cơ chế giám sát sự kiện đa kênh I/O Multiplexing bằng hàm select() được kích hoạt liên tục cho cả hai Socket ID (4 và 5).
- Kết quả:** Khi Client 1 (Terminal 2) truyền chuỗi dữ liệu "Ping", sự kiện FD_ISSET trên Socket 1 được kích hoạt, Server đọc dữ liệu vào vùng đệm buffer rồi chuyển tiếp nguyên bản sang Socket đối tác thông qua hàm send(). Quá trình lặp lại tương tự khi Client 2 phản hồi lại chuỗi "Pong". Độ trễ truyền thông tiệm cận bằng 0, đảm bảo tính toàn vẹn của thông điệp.

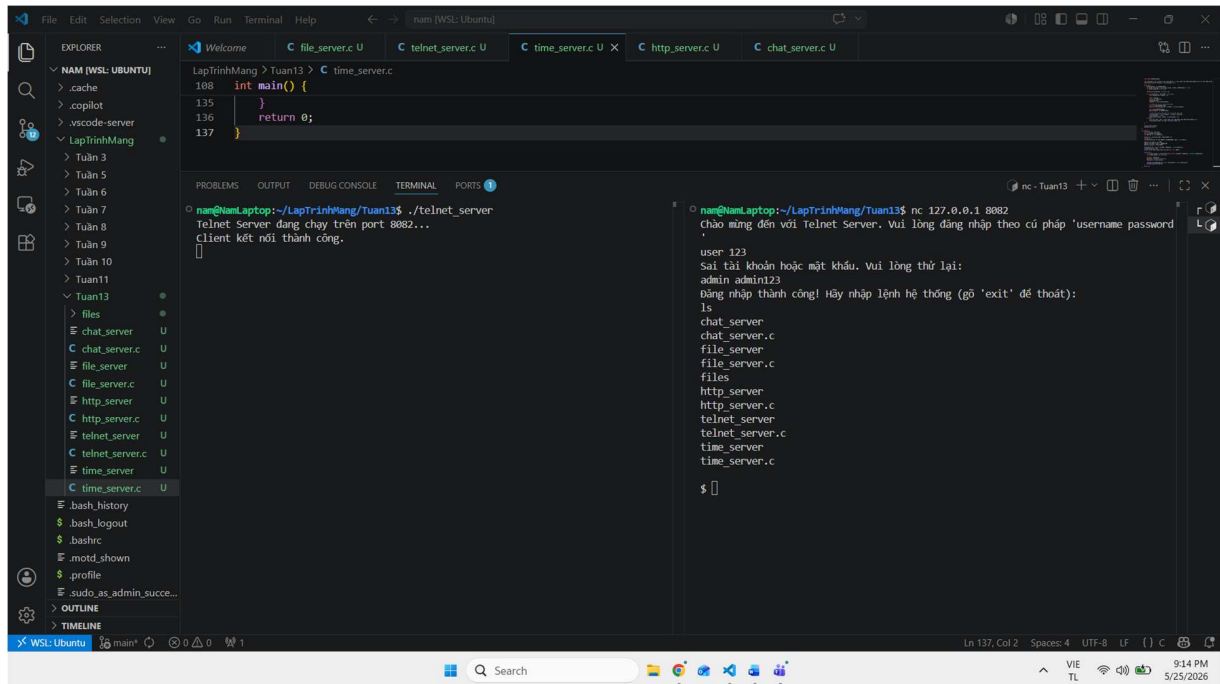


Hình 6. Thu hồi tài nguyên luồng và đóng Socket đồng bộ khi có Client rời phòng.

Giải thích kết quả:

- **Terminal 2 (Client 1):** Người dùng chủ động gửi tín hiệu ngắt kết nối bằng tổ hợp phím Ctrl + C.
- **Kết quả hệ thống:** 1. Hàm `recv()` tại Thread quản lý phiên của Server nhận về giá trị trả về bằng 0 (dấu hiệu ngắt kết nối TCP từ Client 1). 2. Vòng lặp `while(1)` của luồng bị bẻ gãy (`break`). Server lập tức thực hiện lệnh giải phóng tài nguyên chéo bằng cách gọi `close()` đồng thời cho cả Socket của Client 1 và Client 2. 3. Client 2 (Terminal 3) bị ngắt kết nối đồng bộ theo đúng yêu cầu đề bài. 4. Server in log: "Một phiên chat đã kết thúc. Đóng kết nối của cả 2 clients." và giải phóng Thread. Riêng Client 3 (Terminal 4) vẫn nằm trong hàng đợi tổng một cách độc lập và an toàn, hoàn toàn không bị ảnh hưởng bởi sự sập kết nối của cặp chat bên cạnh.

Bài tập telnet_server:

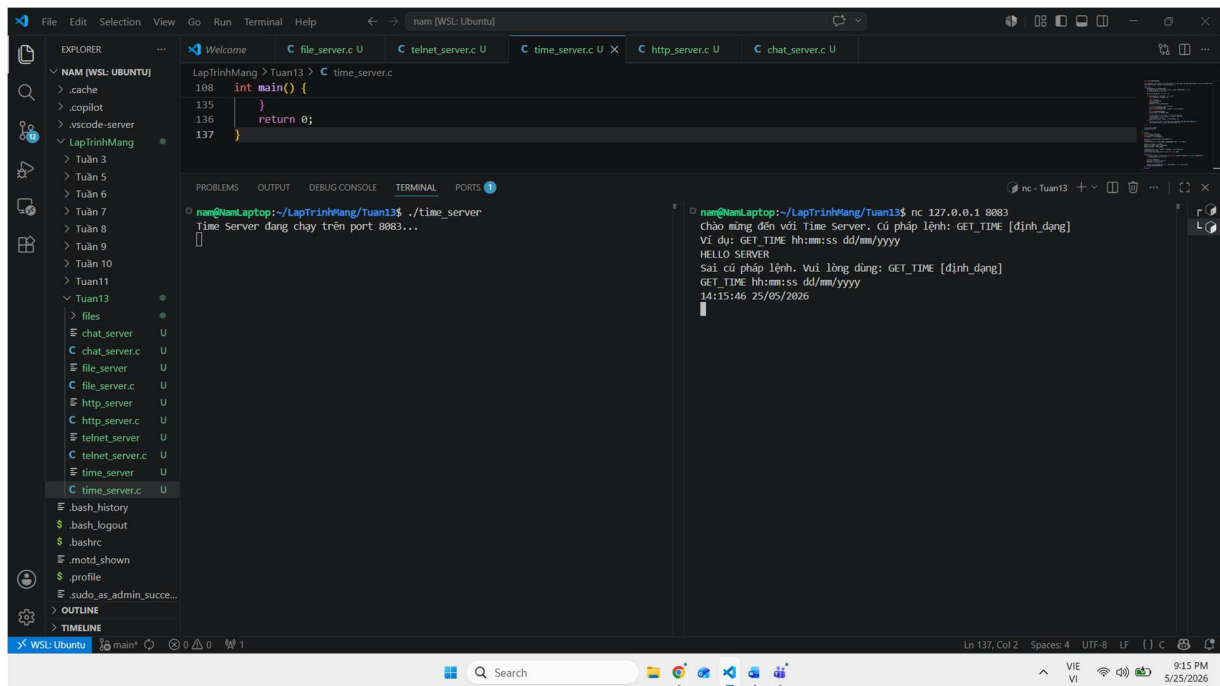


Hình 7. Giao thức xác thực tài khoản và thực thi lệnh hệ thống từ xa qua Telnet Server.

Giải thích kết quả:

- Phía Server:** Lắng nghe tại cổng 8082. Khi nhận được yêu cầu kết nối, tiến trình cha chấp nhận socket và ủy nhiệm toàn quyền xử lý cho một luồng dịch vụ độc lập thông qua hàm `pthread_create()`, đồng thời thực hiện `pthread_detach()` để hệ thống tự động thu hồi tài nguyên luồng khi kết thúc phiên.
- Phía Client:**
 - * Giai đoạn Xác thực:** Khi kết nối bằng nc, Client đầu tiên nhập sai thông tin cú pháp tài khoản (user 123), luồng dịch vụ trên Server kiểm tra điều kiện `!authenticated`, so khớp chuỗi thất bại và gửi trả thông điệp cảnh báo từ chối. Ngay sau đó, Client nhập chính xác thông tin admin admin123, trạng thái luồng chuyển đổi sang `authenticated = 1`.
 - Giai đoạn Thực thi Lệnh:** Client truyền lên lệnh Shell của Linux là `ls` hoặc `pwd`. Luồng dịch vụ của Server đón nhận dữ liệu, gọi hàm hệ thống `popen()` để mở một pipe nhị phân chạy lệnh trực tiếp trên OS của Server. Toàn bộ kết quả đầu ra (STDOUT) từ lệnh hệ thống được đọc tuần tự vào vùng đệm buffer và chuyển tiếp nguyên bản về cho Client thông qua hàm `send()`.

Bài tập time_server:

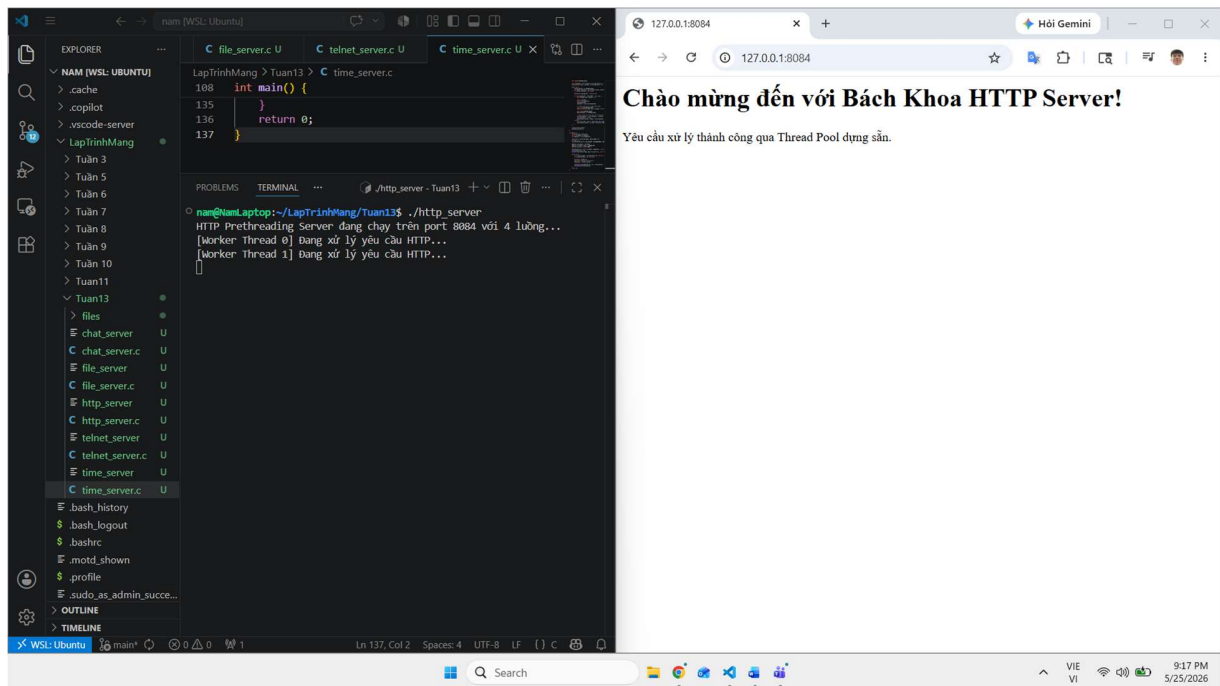


Hình 8. Luồng xử lý phân tích cú pháp định dạng thời gian thực thời gian thực.

Giải thích kết quả:

- **Cơ chế xử lý đồng thời:** Định kiến trúc tương tự mô hình đa luồng (Multi-threading), mỗi phiên kết nối của một Client mới được duy trì độc lập, đảm bảo nhiều Client có thể tra cứu thời gian cùng một lúc mà không gây nghẽn hệ thống.
- **Cơ chế xử lý dữ liệu:** Client gửi yêu cầu lấy thời gian theo cú pháp chuẩn của giao thức: GET_TIME hh:mm:ss dd/mm/yyyy.
- **Kết quả:** Luồng dịch vụ bóc tách chuỗi nhận được từ hàm recv(), trích xuất phần chuỗi định dạng thô phía sau (hh:mm:ss dd/mm/yyyy). Hệ thống gọi hàm convert_format() tự xây dựng để tịnh tiến con trỏ, ánh xạ an toàn các token thô sang cấu trúc định dạng chuẩn của Linux (ví dụ: yyyy đổi thành %Y, dd đổi thành %d). Sau đó, chương trình đọc thời gian thực từ chip đồng hồ của Server thông qua time() và localtime(), chuyển đổi thành chuỗi ký tự bằng strftime() và phản hồi chính xác giá trị thời gian hiện tại về máy Client.

Bài tập http_server:



Hình 9. Mô hình luồng dựng sẵn (Thread Pool) kết hợp Mutex Lock xử lý gói tin HTTP thô.

Giải thích kết quả:

- **Phía Server (Kiến trúc Prethreading):** Thay vì tạo luồng động sau khi accept, ngay khi khởi động, Server khởi tạo sẵn một nhóm gồm 4 luồng làm việc cố định (NUM_THREADS = 4). Cả 4 luồng này đồng thời rơi vào trạng thái block (pthread_cond_wait) để chờ tín hiệu giải phóng từ biến điều kiện khi hàng đợi rỗng.
- **Luồng tương tác:** Khi người dùng mở trình duyệt Web (Chrome/Firefox) truy cập địa chỉ http://127.0.0.1:8084/, luồng chính của Server bắt được sự kiện kết nối, gọi hàm enqueue() để đẩy Socket ID này vào cấu trúc hàng đợi vòng (Circular Queue) toàn cục dưới sự bảo vệ của pthread_mutex_lock nhằm tránh hiện tượng Race Condition.
- **Kết quả:** Lệnh pthread_cond_signal() phát tín hiệu đánh thức, một Worker Thread (ví dụ: Thread 0 hoặc Thread 1 tùy theo cơ chế lập lịch của OS) sẽ giành được quyền kiểm soát Mutex, lấy Socket ID ra khỏi hàng đợi (dequeue()) và xử lý gói tin HTTP Request. Luồng phân tích dòng tiêu đề GET / HTTP/1.1, đóng gói dữ liệu phản hồi theo đúng chuẩn giao thức HTTP Response (bao gồm mã trạng thái 200 OK, Content-Type, Content-Length) kèm nội dung trang HTML, truyền tải thành công giúp trình duyệt hiển thị giao diện văn bản tường minh. Sau khi truyền xong, luồng tự động đóng socket và quay lại trạng thái chờ việc trong Pool.